

Cours 18 — Projet final : peinture par particules

Avant : tout ce qui précède. Chaque étape rappelle un cours. **Comment travailler** : toujours dans `ofApp.h` et `ofApp.cpp`, en construisant étape par étape — les deux premières donnent déjà un résultat. `ofApp18.h` / `ofApp18.cpp` : la référence téléchargeable de l'état final. Les images `pandaroux.jpg`, `redpanda.jpg`, `chibi-redpanda.jpg` vont dans `bin/data`.

Six cents particules se promènent sur la fenêtre. Chacune dépose, à chaque frame, un petit cercle de la couleur du pixel de la photo qui se trouve sous elle. On n'efface jamais : l'image apparaît toute seule, en une vingtaine de secondes, comme une peinture qui se fait.



Le projet est construit en six étapes. Les deux premières suffisent pour avoir un résultat ; les quatre suivantes s'activent et se désactivent avec les touches 3 à 6, pour voir ce que chacune apporte. Espace efface, `i` change d'image, `c` montre les contours.

■ 1. Étape 1 — des particules qui marchent

```
std::vector<float> posX;  
std::vector<float> posY;  
std::vector<float> angle;
```

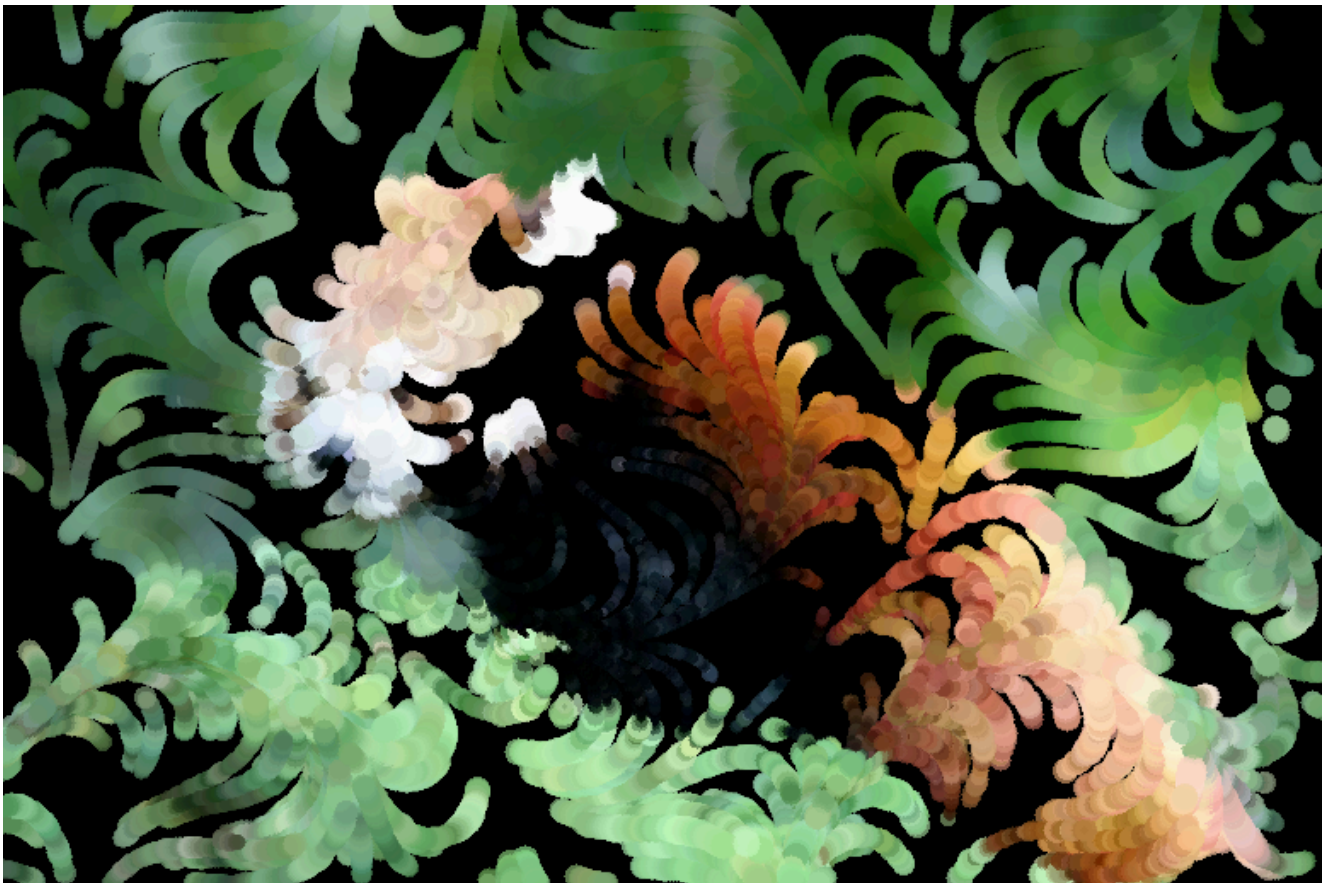
Une particule, c'est une position et une direction. Trois listes parallèles (cours 07), remplies au hasard (cours 03, 04). Chaque frame, la particule avance dans sa direction à `vitesse` pixels par seconde (cours 06) :

```
posX[i] += cos(angle[i]) * v * dt;  
posY[i] += sin(angle[i]) * v * dt;
```

L'angle donne la direction, `cos` et `sin` la transforment en déplacement horizontal et vertical (cours 13). Une particule qui sort de l'image réapparaît n'importe où, pour que les traits restent répartis partout.

Essaie : dès cette étape (dessine les particules en petits cercles blancs pour les voir), joue avec leur nombre et leur vitesse — 100 lentes, 2000 rapides.

■ 2. Étape 2 — une toile qui ne s'efface pas



Depuis le cours 05, chaque frame repart d'un fond propre. Ici on veut l'inverse. La seule nouveauté du projet : un `offbo`, une image **dans laquelle on peut dessiner**.

```
toile.begin();  
// ... tout ce qui est dessiné ici s'accumule dans la toile  
toile.end();  
toile.draw(0, 0);
```

Entre `begin()` et `end()`, les ordres de dessin vont dans la toile au lieu de la fenêtre. Ce qui y est dessiné y reste, frame après frame. Puis on affiche la toile comme une image. Pour effacer, on met un drapeau `effacer` (cours 17) et on appelle `ofClear(0)` dans la toile une seule fois.

À l'intérieur, pour chaque particule : lire la couleur de la source sous elle (cours 10), dessiner un petit cercle de cette couleur. C'est déjà le moment où l'image émerge du noir.

Essaie : laisse tourner deux minutes. Puis implémente l'effacement (espace → drapeau `effacer` → `ofClear(0)` dans la toile) et regarde l'image renaître.

■ 3. Étape 3 — un champ de direction

```
angle[i] = (sin(x / 80.0f) + cos(y / 80.0f)) * PI + t * 0.1f;
```

Au lieu d'une direction au hasard, la direction **dépend de la position**. Deux particules voisines prennent presque le même angle, donc des traits voisins vont dans le même sens et s'enroulent ensemble. C'est le déphasage du cours 15, appliqué à une direction. Le `t * 0.1f` fait dériver lentement le champ pour que les mêmes zones ne soient pas repassées éternellement.

Désactive avec la touche 3 : les traits redeviennent des lignes droites au hasard. Réactive : ils tourbillonnent.

Essaie : change le `80.0f` (l'échelle du champ) — 20 : des tourbillons serrés ; 300 : de grandes vagues lentes.

■ 4. Étape 4 — des traits qui respectent les formes



Au démarrage, on calcule une fois l'image des contours avec le noyau du cours 17, sur la luminance. Elle donne, pour chaque pixel, un nombre entre 0 (zone plate) et 1 (bord net).

```
float bord = contours.getColor(px, py).r / 255.0f;  
v = vitesse * (1 - 0.7f * bord);  
angle[i] += bord * ofRandom(-3, 3) * dt;
```

Sur un bord, la particule ralentit et dévie au hasard : ses traits se brisent en petites touches là où il y a du détail, et restent longs dans les aplats. Dans `draw()`, la taille du cercle suit la même logique : `6 - 5 * bord`, gros dans les aplats, fin sur les bords. Un peintre fait la même chose.



Essaie : le `0.7f` dose le ralentissement sur les bords — pousse-le à `0.95`, ou coupe-le à `0`.

■ 5. Étape 5 — une couleur vivante

```
float decalage = (i % 21) - 10;
float teinte = fmod(c.getHue() + decalage + 255, 255);
c = ofColor::fromHsb(teinte, c.getSaturation(), c.getBrightness());
taille = taille * (0.5f + c.getBrightness() / 255.0f);
```

Chaque particule décale légèrement la teinte du pixel qu'elle lit (cours 09, 11), de `-10` à `+10` selon son numéro. Un aplat de vert uniforme devient un mélange de verts voisins, comme des touches de pinceau. Les zones claires reçoivent des touches plus larges. `ofSetColor(c, 180)` ajoute une légère transparence pour que les passages successifs se mélangent au lieu de se recouvrir.

Essaie : élargis le décalage de teinte (`(i % 21) * 3 - 30`) — à partir de quand la peinture devient-elle fauviste ?

■ 6. Étape 6 — la souris

```
float d = ofDist(mouseX, mouseY, x, y);
if (d < 100 && d > 0) {
    angle[i] = atan2(y - mouseY, x - mouseX);
    v = vitesse * 3;
}
```

Distance du cours 12. Dans un rayon de 100 pixels, la particule fuit : `atan2(dy, dx)` donne l'angle du vecteur qui va de la souris à la particule, c'est l'inverse de `cos` / `sin` (cours 16). Elle prend cette direction et accélère. Passer la souris sur la toile repousse la peinture.

Essaie : inverse le vecteur (`atan2(mouseY - y, mouseX - x)`) — la souris **attire** les particules. Laquelle des deux versions fait le plus « pinceau » ?

■ 7. Le clavier

```
if (key == '3') champDirection = !champDirection;
```

Le `!` (cours 10) inverse un `bool` : `true` devient `false` et réciproquement. Une ligne pour basculer une option. Pour changer d'image :

```
imageCourante = (imageCourante + 1) % images.size();
```

Le `%` (cours 13) ramène à 0 après la dernière : les images défilent en boucle. La fonction `changerImage` recharge la source, adapte la fenêtre, recalcule les contours et relance les particules.

■ Pour aller plus loin

- Un trait au lieu d'un cercle : garder la position précédente et tracer `ofDrawLine`.
- Ta propre image dans `bin/data` et dans la liste `images`.
- Des traits qui suivent vraiment les formes : les deux noyaux de Sobel (cours 17) donnent la direction du contour, `atan2(sobelY, sobelX)`, et la particule prend cet angle plus $\pi / 2$.
- La webcam à la place de la photo (`ofVideoGrabber`) : la peinture suit ce qui bouge.
- Sauvegarder la toile : `toile.readToPixels(pixels); ofSaveImage(pixels, "peinture.png");`.

■ Ce que ce projet récapitule

Étape	Cours
listes de particules, hasard	03, 04, 07

Étape	Cours
déplacement par angle et <code>dt</code>	06, 13
lire la couleur sous la particule	10
champ de direction, déphasage	15
contours calculés une fois, drapeau	17
teinte et luminosité	09, 11
distance à la souris, <code>atan2</code>	12, 16
toile persistante	<code>offbo</code> , la seule nouveauté